Department of Computer Science & Engineering

ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	**CSA1223**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO4**	**Assessment:**	**ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK**
Weightage:	**30%**	**Total Marks:**	**25**
CO4 Statement:	*Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.*		
Student Name:	G.THARAKNATHREDDY	**Reg.No.:**	192472032
Department:	CSE-AI	**Date:**	

ANNEXURE A
INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization
   A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

2. Cloud Server Memory Bottleneck
   A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

3. Autonomous Vehicle Data Processing
   An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

4. AI Inference Edge Device
   An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

5. High-Performance Gaming Console
   A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

***Code of Conduct**:*

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Understanding of Industry Problem (20%)	**Application of Engineering Concepts (25%)**	**Solution Design (25%)**	**Use of Tools (15%)**	**Analysis (10%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints(1)	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem(1.25)	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards(1.25)	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data(0.75)	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation(0.5)	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

ANSWERS:

Q1. Smartphone Performance Optimization

1. Understanding of Industry Problem – 20%

A smartphone may experience lag while switching between applications because the processor frequently waits for data to be transferred between memory levels. Modern smartphones use a **memory hierarchy** consisting of CPU registers, L1 cache, L2 cache, L3 cache, and DRAM. When switching between applications, the processor needs application instructions, user data, operating-system data, and frequently accessed resources. If this information is not available in cache, the CPU must access slower DRAM, increasing latency.

The main requirements are:

- Reduce memory access latency.
- Increase CPU data availability.
- Reduce DRAM accesses.
- Improve application switching speed.
- Maintain reasonable power consumption.
- Provide sufficient memory bandwidth for multitasking.

2. Application of Engineering Concepts – 25%

L1 Cache

L1 is the **smallest and fastest cache** and is located closest to the CPU cores.

- Very low access latency.
- Usually divided into instruction and data caches.
- Stores frequently accessed instructions and data.
- Limited capacity.

L2 Cache

L2 is larger than L1 but slower.

- Stores data that cannot fit into L1.
- Reduces accesses to lower memory levels.
- Can be private to a CPU core or shared by a cluster.

L3 Cache

L3 is generally larger than L2 and has higher latency.

- Often shared among multiple CPU cores.
- Useful for communication between cores.
- Stores frequently reused application and system data.

DRAM

DRAM provides large-capacity main memory.

- Much larger than cache.
- Higher latency than L1/L2/L3.
- Used to store running applications and operating-system data.
- Provides high capacity but consumes more energy than on-chip caches for accesses.

Memory Hierarchy

CPU Registers

↓

L1 Cache

↓

L2 Cache

↓
L3 Cache
↓
DRAM
↓
Storage

As we move downward:

- Capacity increases.
- Cost per bit decreases.
- Latency increases.

3. Solution Design – 25%

A suitable smartphone memory hierarchy is:

CPU
|
L1 Cache
32–64 KB/core
|
L2 Cache
256 KB–1 MB/core
|
L3 Cache
4–16 MB
|
High-Speed DRAM
8–16 GB
|
UFS Storage

Proposed Improvements

1. Increase effective L1 cache utilization

Frequently executed application instructions should remain in L1.

2. Optimize L2 cache

A sufficiently large L2 cache can reduce expensive accesses to L3 and DRAM.

3. Shared L3 cache

A shared L3 cache allows applications and CPU cores to reuse common data efficiently.

4. High-bandwidth LPDDR

Use modern low-power, high-bandwidth LPDDR memory to improve application loading and multitasking.

5. Better prefetching

Hardware and software prefetchers can predict future memory accesses and bring data into cache before it is required.

6. Memory compression

Frequently inactive pages can be compressed to reduce DRAM pressure.

4. Use of Tools – 15%

The design can be evaluated using:

- **Android Studio Profiler** – application memory analysis.
- **Perfetto** – CPU and memory-performance tracing.
- **Linux perf** – cache-miss analysis.

- **ARM Performance Monitor Unit (PMU)** – hardware performance counters.
- **Geekbench/benchmark workloads** – performance comparison.

Important measurements include:

Metric	Purpose
L1 Cache Miss Rate	Measures L1 effectiveness
L2 Cache Miss Rate	Evaluates L2 performance
L3 Cache Miss Rate	Determines DRAM pressure
DRAM Bandwidth	Measures memory throughput
Memory Latency	Measures access delay
Application Switch Time	Measures user-visible improvement

5. Analysis – 10%

The proposed hierarchy reduces the number of accesses reaching DRAM. Frequently accessed data remains in L1/L2/L3, while DRAM provides capacity for larger application datasets. Therefore:

More cache hits → fewer DRAM accesses → lower latency → faster application switching.

The best solution is not simply increasing DRAM capacity. A balanced hierarchy with **efficient L1/L2/L3 caches, high-bandwidth LPDDR DRAM, prefetching, and memory compression** provides better overall performance.

Q2. Cloud Server Memory Bottleneck

1. Understanding of Industry Problem – 20%

A cloud service provider experiences high database-query latency. Database applications perform large numbers of memory accesses, and performance can be affected by:

- CPU cache misses.
- Insufficient DRAM.
- Virtual-memory page faults.
- Disk/SSD accesses.
- Poor database memory allocation.
- Memory contention between virtual machines.

A cache miss is different from a page fault. Cache misses occur within the CPU memory hierarchy, whereas virtual-memory paging moves memory pages between DRAM and secondary storage.

2. Application of Engineering Concepts – 25%

Cache Hierarchy

CPU
↓
L1 Cache
↓
L2 Cache
↓
L3 Cache
↓
DRAM

Caches store recently or frequently accessed data.

Virtual Memory

Virtual memory provides each process with a virtual address space.

Virtual Address

↓

Page Table

↓

Physical DRAM

↓

SSD if page is not in DRAM

If a required page is absent from DRAM, a **page fault** occurs and the operating system may need to retrieve it from storage.

Cache vs Virtual Memory Paging

Feature	**Cache Hierarchy**	**Virtual Memory Paging**
Main purpose	Reduce CPU access latency	Extend/manage memory address space
Unit	Cache line	Page
Fastest level	L1	DRAM
Miss/fault	Cache miss	Page fault
Typical penalty	Low to moderate	Very high
Managed by	Hardware mostly	OS + hardware
Storage involved	Normally no	Potentially SSD

3. Solution Design – 25%

Recommended cloud-server architecture:

CPU

│

L1

│

L2

│

Large L3

│

High-Bandwidth DDR5/Server DRAM

│

NVMe SSD

│

Cloud Storage

Improvements

1. Increase L3 cache capacity where workload justifies it.
2. Use high-bandwidth server DRAM.
3. Increase available DRAM to avoid excessive paging.
4. Optimize database buffer pools.
5. Keep frequently accessed database pages in memory.
6. Use NVMe SSD rather than slower storage for unavoidable paging.
7. Monitor NUMA locality on multi-socket servers.
8. Reduce unnecessary virtual-machine memory contention.

Database Buffer Pool

A database should maintain frequently accessed pages in DRAM:

Database Query
↓
Database Buffer Pool
↓
DRAM
↓
NVMe SSD

This avoids repeatedly reading database blocks from storage.

4. Use of Tools – 15%

Useful tools include:

- Linux perf
- vmstat
- iostat
- Database performance monitors
- Prometheus
- Grafana
- Cloud monitoring services
- NUMA monitoring tools

Measurements:

Metric	Purpose
Cache Miss Rate	CPU efficiency
RAM Utilization	Memory pressure
Page Fault Rate	Paging activity
DRAM Bandwidth	Memory throughput
Query Latency	Database performance
I/O Latency	Storage bottleneck

5. Analysis – 10%

Cache hierarchy and virtual-memory paging solve different problems.

Cache hierarchy should be optimized first because cache misses are much cheaper than storage accesses.

However, if the system continuously runs out of DRAM, paging can cause extremely high latency.

Therefore, the recommended priority is:

Optimize cache → increase/optimize DRAM → reduce paging → use fast NVMe for unavoidable storage accesses.

Q3. Autonomous Vehicle Data Processing

1. Understanding of Industry Problem – 20%

An autonomous vehicle continuously receives data from:

- Cameras.
- LiDAR.
- Radar.
- Ultrasonic sensors.
- GPS.
- IMUs.

This data must be processed in real time. Any significant memory delay can increase the time required for object detection, localization, sensor fusion, and decision-making.
The memory system therefore requires:

- Very low latency.
- High bandwidth.
- Predictable performance.
- High reliability.
- Low power consumption.

2. Application of Engineering Concepts – 25%

SRAM

SRAM is extremely fast and does not require periodic refresh.
Advantages:

- Very low latency.
- High performance.
- Suitable for CPU/GPU caches.

Disadvantage:

- High area cost.
- Lower density.

DRAM

DRAM provides high-density working memory.
Advantages:

- Large capacity.
- Good cost per bit.
- Suitable for sensor-processing workloads.

Disadvantages:

- Higher latency than SRAM.
- Requires refresh.

MRAM

MRAM is non-volatile memory based on magnetic storage.
Advantages:

- Non-volatile.
- Low standby power.
- High endurance.
- Faster than many conventional non-volatile memories.

Comparison

Property	SRAM	DRAM	MRAM
Speed	Very high	High	High
Capacity	Low	High	Medium
Volatile	Yes	Yes	No
Power	Higher area cost	Refresh power	Low standby
Best use	Cache	Main memory	Persistent/fast memory

3. Solution Design – 25%

Recommended hierarchy:
AI/CPU/GPU/NPU
↓

SRAM Cache

↓

High-Speed DRAM

↓

MRAM

↓

Storage

SRAM

Store:

- Frequently accessed sensor-processing data.
- AI intermediate values.
- Critical instructions.
- Small working datasets.

DRAM

Store:

- Camera frames.
- LiDAR point clouds.
- Radar data.
- AI model working memory.
- Large intermediate datasets.

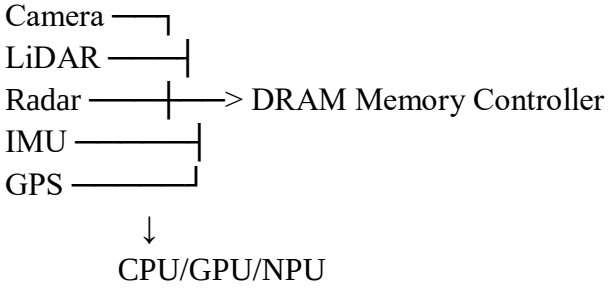
MRAM

Use MRAM for:

- Fast boot data.
- Persistent configuration.
- Critical system state.
- Frequently required persistent data.

Bandwidth Optimization

Multiple memory channels can be used to increase bandwidth.



4. Use of Tools – 15%

Useful tools:

- NVIDIA Nsight for GPU-based systems.
- ARM performance counters.
- Linux perf.
- MATLAB/Simulink.
- ROS/ROS 2 profiling.
- Hardware memory analyzers.

Test parameters:

- Sensor throughput.
- Memory bandwidth.
- Cache miss rate.

- Processing latency.
- Frame-processing time.
- Power consumption.

5. Analysis – 10%

SRAM should handle the most latency-sensitive data, while DRAM should provide high-capacity working memory. MRAM can provide persistent storage with low standby power.
The proposed design provides:
SRAM → minimum latency
DRAM → maximum working capacity and bandwidth
MRAM → non-volatile, low-power persistence
This combination is suitable for real-time autonomous vehicle workloads.

Q4. AI Inference Edge Device

1. Understanding of Industry Problem – 20%

An edge AI device may need to load an AI model quickly while operating under strict power and memory constraints.
Examples include:

- Smart cameras.
- Industrial sensors.
- Drones.
- IoT gateways.
- Robotics systems.

The major challenge is balancing:
speed + capacity + power consumption + persistence.

2. Application of Engineering Concepts – 25%

NAND Flash

NAND Flash is non-volatile and provides high storage capacity.
Used for:

- AI model storage.
- Operating system.
- Application software.
- Firmware.

Its disadvantage is that it is slower than DRAM for active processing.

DRAM

DRAM is volatile and provides fast working memory.
Used for:

- Loading AI models.
- Runtime tensors.
- Intermediate calculations.
- Operating-system processes.

RRAM

RRAM/ReRAM is a resistive non-volatile memory technology.
Potential advantages:

- Non-volatility.
- High density.
- Low standby power.

- Potential suitability for compute-in-memory architectures.

3. Solution Design – 25%

Recommended structure:

AI Accelerator / CPU

↓

On-chip SRAM

↓

High-Speed DRAM

↓

RRAM

↓

NAND Flash

NAND Flash

Store the complete AI models and operating system.

RRAM

Use as a fast non-volatile model/data layer where technology and cost permit.

DRAM

Load active AI models into DRAM during inference.

SRAM

Keep frequently accessed model parameters, activations, and intermediate results close to the accelerator.

Model Loading Process

NAND Flash

↓

RRAM / Fast Persistent Layer

↓

DRAM

↓

SRAM

↓

AI Accelerator

This reduces repeated accesses to slow storage.

4. Use of Tools – 15%

Evaluation can use:

- TensorFlow Lite.
- ONNX Runtime.
- PyTorch Mobile/edge runtimes.
- Linux performance counters.
- Power measurement tools.
- AI accelerator profiling tools.

Measure:

Metric	Goal
Model Loading Time	Reduce
Inference Latency	Reduce
DRAM Bandwidth	Optimize
Energy/Inference	Reduce

Metric	Goal
Memory Usage	Reduce

5. Analysis – 10%

NAND Flash should provide high-capacity persistent storage, while DRAM should provide the main runtime memory. SRAM should handle the most latency-sensitive inference operations. RRAM can provide an additional non-volatile layer and may be especially attractive for future edge-AI architectures that use compute-in-memory.

Therefore:

NAND → capacity
RRAM → non-volatility/low-power fast memory
DRAM → runtime working memory
SRAM → extremely low-latency AI processing

Q5. High-Performance Gaming Console

1. Understanding of Industry Problem – 20%

A gaming console may experience frame drops while loading large scenes because modern games continuously transfer large amounts of:

- Textures.
- 3D models.
- Audio.
- Shader data.
- Animation data.
- World assets.

The CPU/GPU requires high memory bandwidth to process these assets quickly.

If data cannot be supplied quickly enough, the GPU may become idle, causing frame-time spikes and frame drops.

2. Application of Engineering Concepts – 25%

L3 Cache

L3 cache is faster than DRAM and can provide frequently accessed data to CPU cores.

Advantages:

- Lower latency.
- Reduces DRAM accesses.
- Useful for frequently reused game-engine data.

Limitation:

- Relatively small compared with DRAM.
- Cannot hold an entire large game scene.

DRAM

DRAM provides large working capacity.

Advantages:

- Large capacity.
- High bandwidth.
- Suitable for textures and game assets.

Limitation:

- Higher latency than cache.

Comparison

Feature	L3 Cache	DRAM
Capacity	Small	Large
Latency	Low	Higher
Bandwidth	High	Very high
Cost/bit	High	Lower
Main purpose	Frequently reused data	Large working datasets

3. Solution Design – 25%

Recommended architecture:

CPU
│
L1 Cache
│
L2 Cache
│
Large/Shared L3 Cache
│
High-Bandwidth DRAM
│
High-Speed SSD

Improvement 1: Increase L3 cache

A larger shared L3 can improve CPU-side game-engine performance by retaining frequently used data.

Improvement 2: Increase DRAM bandwidth

High-bandwidth memory allows the GPU to receive textures, geometry, and other assets faster.

Improvement 3: Asset Streaming

Instead of loading an entire large scene at once:

SSD
↓
Scene Asset Streaming
↓
DRAM
↓
GPU

Only the assets required for the current scene should be transferred.

Improvement 4: Compression

Texture and geometry compression reduces the amount of data transferred.

Improvement 5: Direct GPU Data Movement

Use optimized DMA/memory-controller mechanisms to reduce unnecessary CPU copying.

4. Use of Tools – 15%

Useful tools include:

- GPU profiling tools.
- CPU performance counters.
- Frame-time analyzers.
- Game-engine profilers.
- Memory bandwidth monitors.
- SSD performance benchmarks.

Important measurements:

Metric	Purpose
Frame Time	Detect frame drops
FPS	Overall gaming performance
L3 Miss Rate	Evaluate cache
DRAM Bandwidth	Evaluate throughput
GPU Utilization	Detect GPU starvation
Asset Loading Time	Evaluate scene loading

5. Analysis – 10%

L3 cache and DRAM serve different purposes. Increasing L3 cache can reduce CPU memory latency, but it cannot replace high-capacity DRAM.

For large scene loading, **DRAM bandwidth is particularly important** because textures and game assets can be very large.

The optimal solution is therefore:

Large/efficient L3 → high-bandwidth DRAM → fast SSD → intelligent asset streaming.

This reduces frame-time spikes and helps maintain stable FPS.